



Práctica Obligatoria 2

Aprendizaje automático para la lectura de paneles informativos en autopistas

Visión Artificial — Grado en Ingeniería de Computadores
Universidad Rey Juan Carlos

Autores: Nicolás Wenceslao Muñoz Ciudad

Jorge Bernabé Molinero

Joel Domené Álvaro

Curso 2025/2026

Índice

1. Introducción y organización del código.....	3
2. Ejercicio 1: clasificador básico (gris + LDA + Normal Bayes).....	4
2.1. Algoritmo.....	4
2.2. Código del clasificador.....	4
2.3. Resultados sobre test_ocr.....	5
3. Ejercicio 2: alternativas y comparativa.....	7
3.1. Métodos de biblioteca empleados.....	8
3.2. Comparativa (validación sobre test_ocr).....	8
4. Ejercicio 3: lectura automática de paneles recortados.....	9
4.1. Código (detección, RANSAC y separación).....	9
4.2. Resultados (distancia de Levenshtein vs. gt.txt).....	11
5. Ejercicio 4: sistema completo (detección P1 + OCR P2).....	12
6. Ejecución y conclusiones.....	14

1. Introducción y organización del código

Esta práctica implementa un sistema de Reconocimiento Óptico de Caracteres (OCR) para leer el texto de los paneles informativos de autopista. Se ha seguido el diseño orientado a objetos propuesto en el enunciado, con una clase base OCRClassifier y clasificadores derivados, organizados en el paquete ocr/.

Bibliotecas utilizadas: OpenCV (cv2), scikit-learn, scikit-image (HOG) y numpy. Los algoritmos estándar empleados se citan en cada apartado indicando la función concreta de la biblioteca.

Estructura de ficheros:

- **ocr/ocr_classifier.py** — clase base: mapeo carácter↔etiqueta, preprocesado común y construcción del dataset.
- **ocr/char_segmentation.py** — umbralizado adaptativo + findContours + boundingRect (segmentación de un carácter).
- **ocr/features.py** — vector de características (niveles de gris / HOG).
- **ocr/lda_normal_bayes_classifier.py** — sistema básico del Ejercicio 1.
- **ocr/classifiers.py** — sistema configurable del Ejercicio 2.
- **ocr/panel_reader.py** — detección de caracteres, RANSAC de líneas y lectura del panel (Ejercicio 3).
- **ocr/data_loader.py** — carga de train_ocr / test_ocr.
- **evaluar_clasificadores_OCR.py** — Ejercicios 1 y 2 (--classifier).
- **main_panels_ocr.py** — Ejercicio 3 (lectura de paneles recortados).
- **main.py + detectors.py** — Ejercicio 4 (detector MSER de la P1 + OCR).

Clasificadores registrados (parámetro --classifier): lda_bayes, lda_knn, lda_svm, lda_euclid, pca_knn, hog_lda_knn, hog_lda_svm, hog_none_svm.

2. Ejercicio 1: clasificador básico (gris + LDA + Normal Bayes)

2.1. Algoritmo

El pipeline es idéntico en entrenamiento y en los paneles reales, de modo que el clasificador ve siempre el mismo tipo de entrada:

- Conversión a gris (cv2.cvtColor).
- Umbralizado adaptativo: cv2.adaptiveThreshold (ADAPTIVE_THRESH_GAUSSIAN_C, THRESH_BINARY_INV). La polaridad se ajusta automáticamente mirando el borde de la imagen, dejando el carácter en blanco sobre fondo negro.
- Localización del carácter: cv2.findContours + cv2.contourArea (mayor contorno) + cv2.boundingRect.
- Recorte y redimensionado a 25×25 (cv2.resize).
- Vector de características: la matriz 25×25 de niveles de gris se convierte en una fila de 625 columnas (np.reshape).
- Reducción de dimensionalidad: sklearn.discriminant_analysis.LinearDiscriminantAnalysis (fit + transform). Con 62 clases, LDA reduce 625 → 61 dimensiones.
- Clasificador: cv2.ml.NormalBayesClassifier (Bayesiano gaussiano). Se convierten las matrices a np.float32 y las etiquetas a np.int32, como exige el módulo ml de OpenCV.

2.2. Código del clasificador

ocr/char_segmentation.py — binarización con ajuste automático de polaridad

```
def binarize(gray, block_size=21, c=10):
    """
    Umbraliza con cv2.adaptiveThreshold y deja SIEMPRE el caracter en blanco
    (255) sobre fondo negro (0), independientemente de la polaridad original.

    - En las imagenes de entrenamiento el texto es oscuro sobre fondo claro.
    - En los paneles de carretera el texto es claro sobre fondo azul oscuro.

    Para unificar ambos casos, tras umbralizar se mira el borde de la imagen
    (que casi siempre es fondo): si el borde queda blanco, se invierte.
    """
    if block_size % 2 == 0:
        block_size += 1
    binary = cv2.adaptiveThreshold(
        gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV, block_size, c)

    h, w = binary.shape
    border = np.concatenate([
        binary[0, :], binary[-1, :], binary[:, 0], binary[:, -1]])
    if border.mean() > 127:
        # el fondo ha quedado blanco -> invertir
        binary = cv2.bitwise_not(binary)
    return binary
```

```
def train(self, images_dict):
    """
    images_dict: diccionario {caracter: [imagenes]}.
    Devuelve (samples, labels) usados en el entrenamiento.
    """
    #1. Caracteristicas (matriz C) y etiquetas (vector E)
    X, y = self.build_dataset(images_dict)      # C y E

    #2. LDA: fit + transform -> matriz reducida CR
    self.lda = LinearDiscriminantAnalysis()
    self.lda.fit(X, y)
    CR = self.lda.transform(X).astype(np.float32) # CR

    #3. Clasificador Normal Bayes de OpenCV (tipos float32 / int32)
    self.classifier = cv2.ml.NormalBayesClassifier_create()
    self.classifier.train(CR, cv2.ml.ROW_SAMPLE,
                          y.astype(np.int32))

    return X, y

def predict(self, img, already_binary=False):
    #Clasifica una imagen de un caracter ya recortado
    feat = self.image_to_feature(img, already_binary).astype(np.float32)
    reduced = self.lda.transform(feat).astype(np.float32)
    _, resp = self.classifier.predict(reduced)
    return int(resp[0, 0])
```

2.3. Resultados sobre test ocr

Conjunto: 62 clases (0-9, a-z, A-Z). Entrenamiento: 625 imágenes/clase (38.750). Validación: test_ocr (15.500 imágenes).

Accuracy = 0,960 · Precisión(macro) = 0,960 · Recall(macro) = 0,960 · F1(macro) = 0,960

La mayoría de clases alcanzan $F1 \approx 1,0$. Los errores se concentran en ambigüedades intrínsecas de los caracteres aislados: O/o/0, l/l/1 y mayúscula vs. minúscula de igual forma (V/v, W/w). Estas confusiones son inevitables sin contexto de palabra y explican casi toda la pérdida de accuracy.

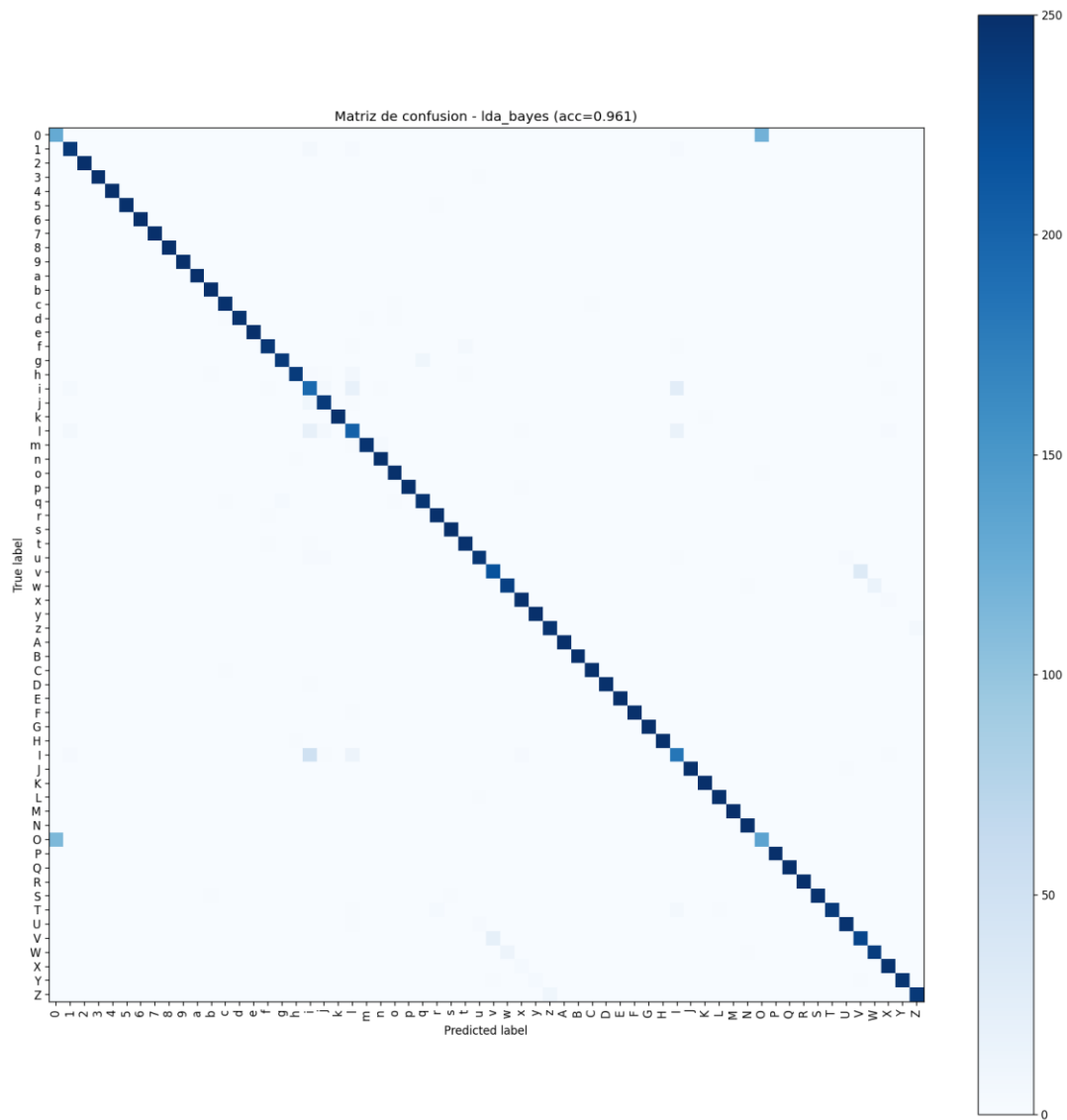


Figura 1. Matriz de confusión del sistema básico (lda_bayes) sobre test_ocr

3. Ejercicio 2: alternativas y comparativa

Se ha implementado un sistema configurable que permite sustituir cualquiera de los tres componentes del sistema básico (extracción de características, reducción de dimensionalidad y clasificador). Cada combinación se selecciona con --classifier.

ocr/classifiers.py — clasificador configurable

```
class GenericOCRClassifier(OCRClassifier):
    #OCR configurable (características + reduccion + clasificador).

    def __init__(self, feature_mode="gray", reducer="lda", clf="knn",
                  ocr_char_size=(25, 25), pca_components=40, knn_k=3):
        super().__init__(ocr_char_size, feature_mode=feature_mode)
        self.reducer_name = reducer
        self.clf_name = clf
        self.classifier_name = "%s_%s_%s" % (feature_mode, reducer, clf)
        self.pca_components = pca_components
        self.knn_k = knn_k
        self.reducer = None
        self.classifier = None

    def _build_reducer(self):
        if self.reducer_name == "lda":
            return LinearDiscriminantAnalysis()
        if self.reducer_name == "pca":
            return PCA(n_components=self.pca_components, whiten=True)
        if self.reducer_name == "none":
            return None
        raise ValueError("Reductor desconocido: %s" % self.reducer_name)

    def _build_classifier(self):
        if self.clf_name == "knn":
            return KNeighborsClassifier(n_neighbors=self.knn_k)
        if self.clf_name == "svm":
            return SVC(kernel="rbf", C=10, gamma="scale")
        if self.clf_name == "euclid":
            return NearestCentroid() # clasificador euclideo
        if self.clf_name == "bayes":
            return GaussianNB()
        raise ValueError("Clasificador desconocido: %s" % self.clf_name)

    def train(self, images_dict):
        X, y = self.build_dataset(images_dict)

        self.reducer = self._build_reducer()
        if self.reducer is not None:
            if self.reducer_name == "lda":
                self.reducer.fit(X, y)
            else:
                self.reducer.fit(X)
            Xr = self.reducer.transform(X)
        else:
            Xr = X

        self.classifier = self._build_classifier()
```

```

self.classifier.fit(Xr, y)
return X, y

def predict(self, img, already_binary=False):
    feat = self.image_to_feature(img, already_binary)
    if self.reducer is not None:
        feat = self.reducer.transform(feat)
    return int(self.classifier.predict(feat)[0])

```

ocr/classifiers.py — registro de clasificadores disponibles

```

AVAILABLE_CLASSIFIERS = {
    #Sistema basico (Ejercicio 1)
    "lda_bayes": lambda: LdaNormalBayesClassifier(),
    #Alternativas (Ejercicio 2)
    "lda_knn": lambda: GenericOCRCClassifier("gray", "lda", "knn"),
    "lda_svm": lambda: GenericOCRCClassifier("gray", "lda", "svm"),
    "lda_euclid": lambda: GenericOCRCClassifier("gray", "lda", "euclid"),
    "pca_knn": lambda: GenericOCRCClassifier("gray", "pca", "knn"),
    "hog_lda_knn": lambda: GenericOCRCClassifier("hog", "lda", "knn"),
    "hog_lda_svm": lambda: GenericOCRCClassifier("hog", "lda", "svm"),
    "hog_none_svm": lambda: GenericOCRCClassifier("hog", "none", "svm"),
}

```

3.1. Métodos de biblioteca empleados

- skimage.feature.hog — descriptor HOG (9 orientaciones, celdas 5×5).
- sklearn.decomposition.PCA — reducción alternativa a LDA (pca_knn); también se permite la ausencia de reducción ('none').
- sklearn.neighbors.KNeighborsClassifier — KNN multiclase (k=3).
- sklearn.neighbors.NearestCentroid — clasificador euclídeo.
- sklearn.svm.SVC — SVM multiclase, kernel RBF.

3.2. Comparativa (validación sobre test ocr)

Clasificador	Características	Reducción	Clasificador	Accuracy
lda_bayes (base)	gris (625)	LDA	Normal Bayes	0,960
lda_knn	gris (625)	LDA	KNN (k=3)	0,959
hog_lda_svm	HOG	LDA	SVM (RBF)	0,958
lda_euclid	gris (625)	LDA	Euclídeo	0,923

Discusión:

Con un conjunto de entrenamiento grande y limpio casi todos los sistemas superan el 95 %. El sistema básico (LDA + Normal Bayes) y LDA+KNN son prácticamente equivalentes y los mejores. El clasificador euclídeo (NearestCentroid) es el peor (0,923) porque asume clases esféricas e isotrópicas, hipótesis que no se cumple tras la proyección LDA. HOG no mejora a los niveles de gris: los caracteres ya están centrados y normalizados, por lo que el gradiente no aporta discriminación adicional. Se mantiene LDA + Normal Bayes como mejor sistema y se usa en los Ejercicios 3 y 4.

4. Ejercicio 3: lectura automática de paneles recortados

El script `main_panels_ocr.py` entrena el clasificador con `train_ocr` y procesa cada panel recortado de `test_ocr_panels`. Algoritmo:

- **Detección de caracteres:** realce de contraste con CLAHE (`cv2.createCLAHE`) y umbralizado de Otsu (`cv2.threshold + THRESH_OTSU`). Se eligió Otsu frente a `adaptiveThreshold` porque conserva trazos sólidos y la topología de los caracteres (huecos de 0, A, e), manteniendo la correspondencia con el entrenamiento. Componentes conexas (`cv2.connectedComponentsWithStats`) y filtrado por altura, anchura, área, relación de aspecto y solidez; se descartan las componentes pegadas al borde (marco, flechas).
- **Líneas de texto:** RANSAC propio sobre los centros de los caracteres (recta $y = m \cdot x + b$). Se separan los inliers como una línea y se itera con el resto; las líneas se ordenan de arriba a abajo y, dentro de cada una, de izquierda a derecha (orden de lectura).
- **Separación de caracteres pegados:** las cajas anchas se dividen según los valles de la proyección vertical; el ancho de carácter se estima con las cajas que ya parecen un único glifo.
- **Clasificación:** cada carácter (ya binarizado, misma representación que en entrenamiento) se clasifica con el mejor sistema del Ej.2. Se genera un único string por panel, sin espacios y con '+' como separador de línea, y se vuelca a `resultado.txt` con el formato `<archivo>;0;0;<W>;<H>;1;1;<texto_ocr>`.

4.1. Código (detección, RANSAC y separación)

ocr/panel_reader.py — detección de caracteres

```
def detect_chars(self, img):
    """
    Devuelve (boxes, mask) donde mask es la mascara binaria del panel
    (caracter=255, fondo=0) y boxes la lista de cajas de caracteres
    candidatos obtenidas con componentes conexas.
    """
    gray = to_gray(img)
    H, W = gray.shape

    #Realce de contraste (los paneles tienen poco contraste)
    gray = cv2.createCLAHE(2.0, (8, 8)).apply(gray)

    #Umbralizado de Otsu
    _, mask = cv2.threshold(
        gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    # El texto siempre es minoritario en un panel -> que sea el frente.
    if mask.mean() > 127:
        mask = cv2.bitwise_not(mask)

    #Apertura mínima para eliminar motas de ruido (sin tapar huecos).
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (2, 2))
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

    n, _, stats, _ = cv2.connectedComponentsWithStats(mask, 8)

    boxes = []
    min_h = self.min_h_ratio * H
```

```

max_h = self.max_h_ratio * H
panel_area = float(H * W)
for i in range(1, n):
    x, y, w, h, area = stats[i]
    if h < min_h or h > max_h:
        continue
    if w < 2 or w > 0.5 * W:
        continue
    if (w * h) > 0.18 * panel_area: #Descarta marco
        continue
    if area < 12: #Descarta motas de ruido
        continue
    #descarta componentes pegadas al borde (marco, flechas, exterior)
    if x <= 1 or y <= 1 or x + w >= W - 1 or y + h >= H - 1:
        continue
    ar = w / float(h)
    if ar < 0.05 or ar > 2.4: #relacion de aspecto de caracter
        continue
    fill = area / float(w * h + 1e-6)
    if fill < 0.10 or fill > 0.98: #Descarta lineas/ruido y bloques
        continue
    boxes.append((x, y, w, h))

return boxes, mask

```

ocr/panel_reader.py — RANSAC de líneas de texto

```

def _ransac_line(points, thr, iters=200):
    #RANSAC para una recta y = m*x + b. Devuelve mascara de inliers
    best_inliers = np.zeros(len(points), dtype=bool)
    n = len(points)
    if n < 2:
        return np.ones(n, dtype=bool) if n == 1 else best_inliers
    xs = points[:, 0]
    ys = points[:, 1]
    rng = np.random.default_rng(42)
    for _ in range(iters):
        i, j = rng.choice(n, 2, replace=False)
        if xs[j] == xs[i]:
            continue
        m = (ys[j] - ys[i]) / (xs[j] - xs[i])
        b = ys[i] - m * xs[i]
        dist = np.abs(m * xs - ys + b) / np.sqrt(m * m + 1.0)
        inliers = dist < thr
        if inliers.sum() > best_inliers.sum():
            best_inliers = inliers
    return best_inliers

```

ocr/panel_reader.py — separación de caracteres pegados por proyección vertical

```

def split_wide_box(binary, box, char_w):
    """
    En los paneles los caracteres de una palabra suelen quedar pegados
    tras umbralizar (una sola componente conexas por palabra). Si la caja
    es mucho mas ancha que un caracter, se divide usando la proyeccion
    vertical (valles = separaciones entre caracteres).
    """
    x, y, w, h = box
    char_w = max(4.0, char_w)
    n = int(round(w / char_w))

```

```

if n <= 1:
    return [box]

region = binary[y:y + h, x:x + w]
col = (region > 0).sum(axis=0).astype(np.float32)

cuts = [0]
for k in range(1, n):
    center = int(k * w / n)
    win = max(2, int(0.35 * w / n))
    a = max(cuts[-1] + 2, center - win)
    b = min(w - 2, center + win)
    if b <= a:
        cut = center
    else:
        cut = a + int(np.argmin(col[a:b])) #Valle de proyeccion
    cuts.append(cut)
cuts.append(w)

sub = []
for i in range(len(cuts) - 1):
    sx, ex = cuts[i], cuts[i + 1]
    if ex - sx < 2:
        continue
    sub.append((x + sx, y, ex - sx, h))
return sub if sub else [box]

```

4.2. Resultados (distancia de Levenshtein vs. gt.txt)

74 paneles evaluados. Distancia media = 10,1. Distancia ≤ 5 : 27/74 (36 %). Distancia ≤ 10 : 40/74 (54 %).

Ejemplos: '500m' $\rightarrow$ '5oom'; 'Monterrel+Verin+Vilaza' $\rightarrow$ 'NoWterrel+Vermn+MIllaza'; '...Madrid459' $\rightarrow$ '...Wadrud45n'. El sistema recupera correctamente la estructura de líneas y la mayor parte de los caracteres; los errores residuales provienen de letras pegadas que la proyección no separa perfectamente y de las confusiones de caso (O/o/0, l/l/1) ya vistas en el Ejercicio 1.

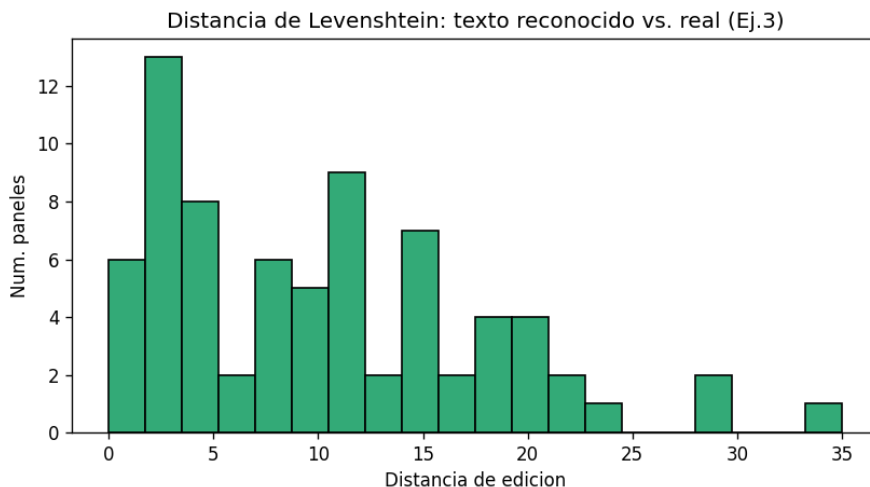


Figura 2. Distribución del error de edición entre el texto reconocido y el real de los 74 paneles.

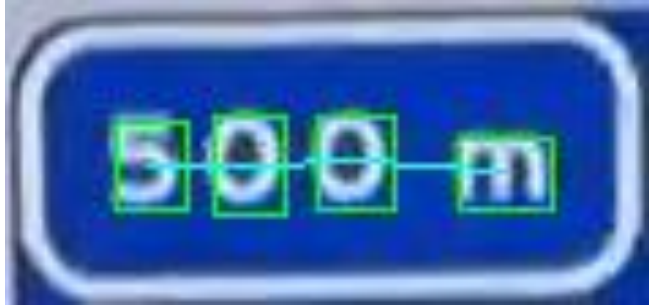


Figura 3. Detección de caracteres (verde) y líneas de texto agrupadas por RANSAC (amarillo) en un panel recortado.



Figura 4. Otro ejemplo de detección y agrupación en líneas.

5. Ejercicio 4: sistema completo (detección P1 + OCR P2)

El script `main.py` une el detector de paneles de la Práctica 1 (`MSERBluePanelDetector`, módulo `detectors.py`) con el lector OCR de esta práctica. Para cada imagen de carretera el detector `MSER` devuelve las cajas de panel; cada panel se recorta y se pasa a `PanelReader.read_annotated()`, reutilizando íntegramente el pipeline del Ejercicio 3. El texto reconocido se escribe como octavo campo de `resultado.txt`: `nombre;x1;y1;x2;y2;1;score;texto_ocr`.

El parámetro `--visualize_ocr` (por defecto `False`) muestra la imagen procesada, los paneles detectados con su score (rojo/amarillo), la caja de cada carácter (verde), las líneas de texto (cian) y el carácter reconocido sobre cada caja (magenta). Se avanza con una tecla y `ESC` termina.

main.py — integración: por cada panel detectado se ejecuta el OCR

```
for (x1, y1, x2, y2, score) in detections:
    crop = img[y1:y2, x1:x2]
    if crop.size == 0:
        annotations.append(("", []))
        text = ""
    else:
        text, ann = reader.read_annotated(crop)
        annotations.append((text, ann))
    fout.write("%s;%d;%d;%d;%d;1;%.4f;%s\n"
              % (name, x1, y1, x2, y2, score, text))
```

Ejecución:

```
python main.py --detector mser \  
  --test_path <dir_test_practica1> \  
  --ocr_classifier lda_bayes --ocr_train_path ./train_ocr \  
  --visualize_ocr
```

Resultado sobre el test de la Práctica 1 (102 imágenes, 161 detecciones con texto OCR). Ejemplos reales: 'TÚNEL DE LA ESCRITA' → 'TUNELDE+LAEscRjTA'; '...Benavente187...Madrid454' → '...Benavente187+Nadrid454'; '500m' → '5oom'.



Figura 5. Sistema completo: dos paneles detectados (rojo) con su score, caracteres (verde), líneas (cian), carácter reconocido (magenta) y texto OCR (amarillo).



Figura 6. Detección + lectura de un panel de destinos.

6. Ejecución y conclusiones

Comandos de ejecución:

```
#Ejercicios 1 y 2 (entrenar + validar un clasificador)
python evaluar_clasificadores_OCR.py --classifier lda_bayes \
    --train_path ./train_ocr --validation_path ./test_ocr

#Ejercicio 3 (lectura de paneles -> resultado.txt)
python main_panels_ocr.py --classifier lda_bayes \
    --train_path ./train_ocr --test_path ./test_ocr_panels
python evaluar_resultados_test_ocr_panels.py \
    --panels_path ./test_ocr_panels

#Ejercicio 4 (deteccion P1 + OCR P2)
python main.py --detector msr --test_path <dir_test_p1> \
    --ocr_classifier lda_bayes --ocr_train_path ./train_ocr \
    --visualize_ocr
```

Conclusiones:

- El clasificador de caracteres alcanza un 96 % de acierto; el error residual es prácticamente solo ambigüedad de caso (O/o/0, I/l/1).
- Entre las alternativas, LDA+Normal Bayes y LDA+KNN son los mejores; el euclídeo es el peor y HOG no mejora a los niveles de gris con estos datos ya normalizados.
- El lector de paneles recupera bien la estructura de líneas (RANSAC) y la mayoría de los caracteres pese a la gran diferencia de dominio entre los caracteres sintéticos de entrenamiento y el texto real, de baja resolución y con letras pegadas, de los paneles.
- El Ejercicio 4 integra el detector MSER de la Práctica 1 con el lector OCR: el sistema completo localiza el panel y lee su contenido en un único paso.
- Mejoras propias: ajuste automático de polaridad en el umbralizado, uso de Otsu+CLAHE para conservar la topología del carácter, separación de caracteres pegados por proyección vertical con ancho auto-estimado, y filtrado de componentes por borde y por outliers de altura dentro de cada línea.